databricks Academy

Lecture - Document Parsing and Chunking Strategies for AI Search

Duration: 30 minutes

Overview

Before a retrieval agent can answer questions about your documents, those documents must be transformed from raw files into searchable, structured data. This lecture covers the key Databricks AI Functions that power this pipeline: `ai_parse_document` for extracting text from PDFs, `ai_classify` and `ai_extract` for structuring content, and `ai_prep_search` for chunking text for AI Search or leveraging reverse ETL for Lakebase Search integration. We'll also cover how chunks flow into an AI Search index: what happens when you create one, how it stays in sync, and why the text you embed differs from the text you retrieve.

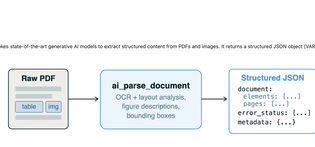
Learning Objectives

By the end of this lecture, you will be able to:

1. Describe how `ai_parse_document` extracts structured content from PDFs
2. Explain the role of `ai_classify` and `ai_extract` in document processing
3. Use `ai_prep_search` to chunk text for AI Search
4. Compare different chunking strategies and their trade-offs
5. Explain what happens when an AI Search index is created and how it stays in sync
6. Distinguish between `chunk_to_embed` and `chunk_to_retrieve` and why they differ

A. The Document Processing Pipeline

A1. From Raw Files to Searchable Data



▼ How does the document processing pipeline work?

RAG systems are only as good as the data they retrieve. Before documents can power a RAG agent, they must be processed through a pipeline that extracts, classifies, and chunks the content. Databricks Knowledge Assistant can ingest UC Volumes and tables directly. It runs its own parsing and chunking pipeline under the hood, so you don't need to wire these AI Functions yourself.

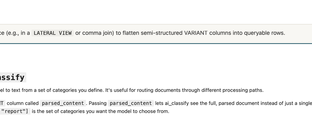
Databricks provides a set of AI Functions (**AWS** | **Azure** | **GCP**), SQL-callable functions powered by foundation models, that handle each stage. As an example workflow, we might have:

- `ai_parse_document` produce a `VARIANT` output
- `ai_prep_search` , `ai_classify` , and `ai_extract` run **in parallel** off that parsed document
- Join the classifications and extracted fields onto the chunk table as extra metadata columns for filtering or ranking in AI Search

B. Parsing Documents with `ai_parse_document`

B1. How It Works

`ai_parse_document` (AWS | Azure | GCP) invokes state-of-the-art generative AI models to extract structured content from PDFs and images. It returns a structured JSON object (VARIANT type) with layout-aware text extraction.



▼ What are the four stages of a Knowledge Assistant pipeline?

Databricks AI Functions transform raw PDFs into searchable data in three stages:

- **Parse (ai_parse_document)** extracts text and layout as structured JSON
- **Classify + Extract (ai_classify, ai_extract)** categorize and pull structured fields
- **Chunk (ai_prep_search)** splits text into semantic chunks for AI Search.
- `version` is an option flag that tells `ai_parse_document` which schema/output version to use.
- **v2 ('2.0')** is the recommended version: it gives you the newer output format (HTML tables, improved layout, etc.) and is what v2 `ai_extract` / `ai_classify` examples are tuned for.

```
SELECT
  path,
  ai_parse_document(content, MAP('version', '2.0')) AS parsed_content
FROM READ_FILES('/Volumes/.../', format => 'binaryFile')
```

The output includes `document:elements` (individual content blocks) and `document:pages` (page-level breakdown), giving you flexible access to the parsed content.

C. Classifying and Extracting with AI Functions

Next, we will look at some examples of how to use `ai_classify()` and `ai_extract()`. Parsing the raw PDF into a structured JSON is straightforward as indicated above. However, we now have to use the table-valude function `variant_explode` (AWS | Azure | GCP) to un-nest our parsed document.

▼ How does variant_explode work?

The `variant_explode` table-valued function turns a `VARIANT` array or object into a set of rows with three columns:

- `pos` `INT` — the position of the element within the array or object
- `key` `STRING` — the field name (objects only; `NULL` for arrays)
- `value` `VARIANT` — the field value or array element

Behavior depends on the input shape:

- **VARIANT object** → `key` and `value` are the object's keys and values.
- **VARIANT array** → `key` is always `NULL` ; `value` holds each element.
- **NULL or non-VARIANT-array/object input** → **no rows are produced**. To preserve the row with `NULL` values instead, use variant_explode_outer (**AWS** | **Azure** | **GCP**).

```
SELECT pos, key, value
FROM variant_explode(parse_json('{"name": "Ada", "age": 36}'));

-- pos | key  | value
-- ----+------+---------
--  0  | name | "Ada"
--  1  | age  | 36
```

Use `variant_explode` as a lateral source (e.g., in a `LATERAL VIEW` or comma join) to flatten semi-structured VARIANT columns into queryable rows.

C1. Categorize Content: `ai_classify`

`ai_classify` (AWS | Azure | GCP) assigns a label to text from a set of categories you define. It's useful for routing documents through different processing paths.

- `ai_parsed_document` produces a `VARIANT` column called `parsed_content` . Passing `parsed_content` lets ai_classify see the full, parsed document instead of just a single text string.
- `["invoice", "receipt", "contract", "report"]` is the set of categories you want the model to choose from.



Sample Code:

```
WITH elements AS (
  &lt;read_file_logic&gt; t,
  LATERAL variant_explode(parsed:document:elements) AS element
  &lt;additional_WHERE_logic&gt;
),
classified AS (
  SELECT
    content,
    ai_classify(
      content,
      '["property_description","pricing_information","location_details","amenities","host_information","other"]'
    ) AS cls
  FROM elements
)
SELECT
  content,
  cls:response[0] AS category -- Parse the results from ai_classify()
FROM classified
```

C2. Pull Structured Fields: `ai_extract`

`ai_extract` (AWS | Azure | GCP) extracts specific fields from unstructured text. You define what to extract, and the AI model finds the values.

Sample Code:

```
WITH elements AS (
  &lt;read_file_logic&gt; t,
  &lt;variant_explode_logic&gt;
),
extracted AS (
  SELECT
    content,
    ai_extract(
      content,
      '["invoice_id", "date", "total"]'
    ) AS ext
  FROM elements
)
SELECT
  content,
  ext:response.invoice_id  AS invoice_id,
  ext:response.date    AS date,
  ext:response.total   AS total
FROM extracted
LIMIT 15
```

D. Chunking with `ai_prep_search`

D1. What is `ai_prep_search` ?

`ai_prep_search` (AWS | Azure | GCP) is a Databricks AI Function that **chunks text for AI Search**. Instead of manually splitting text by character count or using custom chunking logic, `ai_prep_search` uses AI models to split text into semantically meaningful chunks.

Sample Code:

```
SELECT ai_prep_search(parsed_text) AS chunks
FROM parsed_documents
```

D2. Chunking Strategy Considerations

Regardless of the chunking method, keep these principles in mind:

- **Chunk size vs. embedding model limits** - chunks must fit within the embedding model's context window (e.g., 512 tokens for many models). Text beyond the limit is silently truncated.
- **Overlap** - including a small overlap between consecutive chunks (e.g. 10-20%) prevents information from being lost at chunk boundaries.
- **Granularity trade-off** - smaller chunks are more precise for retrieval but may lack surrounding context. Larger chunks capture more context but may dilute the signal.

Lost in the middle

The "Lost in the Middle" phenomenon means LLMs tend to overlook information buried deep in long contents. Creating smaller, focused chunks ensures critical details aren't missed during retrieval.

D3. Embed vs. Retrieve: Why They're Different Columns

From `ai_prep_search` we can produce two distinct columns from the same parsed content. Understanding this separation is important before you create an AI Search Index.

▼ Why not use the same text for both?

Embedding and retrieval serve different goals:

- **Embedding** needs to answer: *"What is this chunk about?"* Adding surrounding context (headers, section titles) helps the embedding model place the chunk in the right region of vector space, even if the chunk text alone is ambiguous. A chunk that says "The fee is $50" is meaningless without context — but prepending "Cleaning Policy > Late Checkout" makes it findable.
- **Retrieval** needs to answer: *"What should the LLM read?"* The LLM needs the original text to generate accurate answers. Prepending metadata here would waste context window tokens and could confuse the model with formatting artifacts.

This separation is a key retrieval quality technique: **optimize for findability when embedding, optimize for readability when retrieving.**

Genie AI CODE

Want to explore more about document parsing and chunking strategies? Ask Genie Code. Click on the genie icon and begin querying. For example, click the **Copy** button below and paste into **Genie Code**.

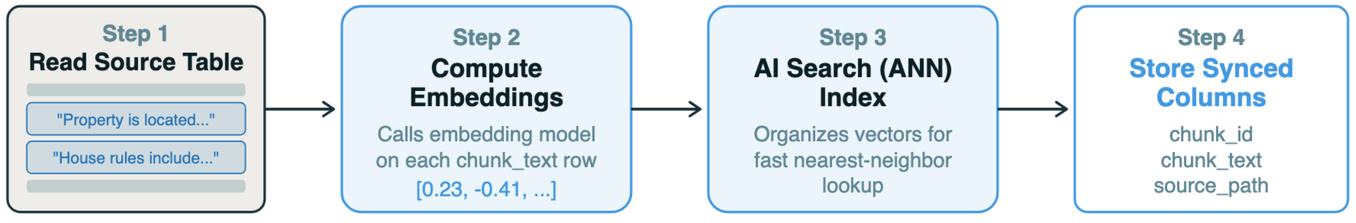
```
How can I combine custom parsing strategies with AI Functions?
```

Copy

E. From Chunks to AI Search Index

E1. What Happens When You Create an Index

Once your chunks are in a Delta table, you create an AI Search index to make them searchable.



Parameter to pay attention when creating a VS endpoint

Parameters	What It Does
`endpoint_name`	Which VS endpoint serves the index (shared compute in this course)
`source_table_name`	The Delta table containing your chunks
`index_name`	Fully-qualified name for your index in Unity Catalog
`pipeline_type`	`TRIGGERED` (sync on demand) or `CONTINUOUS` (sync automatically)
`primary_key`	Unique row identifier used to track which rows have been processed
`embedding_source_column`	Which column to send to the embedding model
`embedding_model_endpoint_name`	The model that converts text to vectors (e.g., `databricks-gte-large-en`)
`columns_to_sync`	Which columns to store alongside vectors so they can be returned in search results

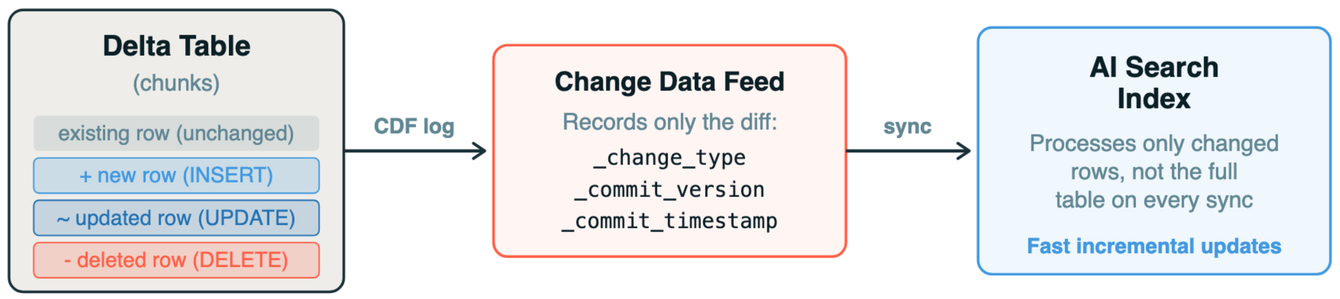
This initial sync can take several minutes depending on the number of rows and chunk sizes.

E2. Keeping the Index in Sync with Change Data Feed

When you write chunks to a Delta table for AI Search, you'll enable **Change Data Feed (CDF)**:

```
ALTER TABLE chunks_table SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
```

For Delta Sync indexes on standard endpoints, CDF is required and is what enables incremental sync behavior. With CDF enabled, AI Search processes only rows changed since the last sync, rather than rebuilding the index the way storage-optimized endpoints partially do on each sync.



Sync modes control *when* those changes flow to the index:

Mode	Behavior	Trade-off
`TRIGGERED`	Syncs only when you explicitly call `index.sync()` or through the UI	You control when updates happen, which is good for batch workflows where you update chunks periodically
`CONTINUOUS`	Syncs automatically as the source table changes	Near-real-time freshness, but uses more compute

F. Other AI SQL Functions

To see the full suite of AI functions, you can view the latest list of task-specific AI functions here (AWS | Azure | GCP).

© 2026 Databricks, Inc. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Privacy Policy | Terms of Use | Support